

Applied Data Science

Semester Project 2025: Web Crawling & Scraping Supreme Court Decisions

May 2025

Nikos Mitsakis
Maria Schoinaki

1. Approach Description

We developed a **crawler** and **scraper** for this project targeting the **Supreme Court of Greece's website**. The main objective was to create a **DataFrame** that fully represents each court decision, meeting both the requirements of the **assignment** and aligning with the structure of the **GreekLegalSum** (*DominusTea*) dataset, which served as a reference point. The goal was to extract structured data from legal decisions, including metadata (*decision number, year, section type, section number*), judges, court intro, law thinking, law reasoning, articles (*4 fields, articles and codes*), full text, full text cleaned, url. We used **Python** with **requests**, **BeautifulSoup** for scraping, and **regex** for precise pattern matching of legal articles. A robust cleaning function ensures normalization of the extracted text. The scraper gracefully handles **HTTP errors**, **timeouts**, and **site encoding** (*Windows-1253 / ISO-8859-7*). Data were stored in a pandas **DataFrame** for further analysis.

2. Specifications

- **Environment:** Python 3.10/3.12
- **Libraries:** requests, BeautifulSoup, re, regex, pandas, matplotlib, seaborn, wordcloud, numpy
- **Encodings handled:** 'windows-1253', fallback to 'utf-8' and 'iso-8859-7'
- **Regex Matching:** Tailored for Greek legal patterns, covering multi-article references, ranges (e.g., *άρθρα 591 έως 681 Δ*), and various formats of codes (e.g., *ΚΠολΔ, ΑΚ, ΠΚ*).

3. Crawling & Scraping Procedure

1. **Collect Decision Links:** We first gathered all decision URLs, paginated them through the site, and validated the links.
2. **Scrape Each Decision:**
 - Extract metadata via regex (*decision number, year, section type/number, judges*).
 - Parse structured sections: court intro, law reasoning, law reasoning.
 - Extract legal article mentions using a comprehensive regex pattern with multiple capture groups (*articles, code, ranges, parenthesis forms, etc.*).
3. **Error Handling:** The crawler skips invalid or missing decisions gracefully and logs any parsing anomalies.
4. **Storage:** Data were compiled into a DataFrame for subsequent statistical and textual analysis.

4. Results Analysis

The dataset includes 2460 (2459 because of 631 decision that is the same as 632) decisions across political and criminal sections. Key insights:

- **Distribution:** Balanced between Political (Πολιτικές) and Criminal (Ποινικές) cases ($\sim\frac{1}{3}$ Criminal, $\sim\frac{2}{3}$ Political), with notable peaks in subsection B1.
- **Legal References:** Most frequent articles are from ΚΠολΔ (e.g., 559, 561, 577) and ΑΚ.
- **Visuals:** Generated count plots, pie charts, heatmaps (e.g., section vs subsection frequency), and word clouds of legal reasoning.
- **Challenges:** Some pages had unusual formatting, requiring tweaks in regex and encoding logic. Certain decisions lacked expected sections (intro/reasoning), leading to missing values, which were accounted for in the analysis. Certain decisions also took significantly more time to scrape due to their size and irregular formatting.

The submitted DataFrame fully meets all field requirements described in the assignment, ensuring a complete and structured representation of each legal decision. Both the CSV file and the Jupyter notebook with the full scraping and analysis code are provided, fulfilling all deliverable criteria. The analysis and interpretation are thoroughly documented within the notebook using markdown cells for clarity.

The CSV file can be found [here](#)